

FutureKawa

Documentation Base de Données — Exploitation

Équipe : Rudy · Florian · Noé · Josselin

Version : 1.0

Date : 02/07/2026

Projet : FutureKawa

Sommaire

Base de Données Exploitation — FutureKawa

01 — Vue d'ensemble

02 — Auth & Utilisateurs

03 — Entrepôts, Zones & Capteurs

04 — Lots & Mouvements

05 — Mesures IoT

06 — Alertes & Incidents

07 — Configuration & Tokens

08 — Schéma des relations

09 — Dictionnaire de données

Base de Données Exploitation — FutureKawa

Projet	FutureKawa
Livrable	Documentation BDD Exploitation
Périmètre	API Django de l'instance exploitation (capteurs, entrepôts, lots, alertes, incidents)
SGBD	PostgreSQL 15
ORM	Django ORM + migrations
Équipe	Rudy · Florian · Noé · Josselin

Sommaire

Section	Fichier
Vue d'ensemble	01-vue-ensemble.md
Auth & utilisateurs	02-auth-utilisateurs.md
Entrepôts, zones & capteurs	03-entrepots-zones-capteurs.md
Lots & mouvements	04-lots-mouvements.md
Mesures IoT	05-mesures.md
Alertes & incidents	06-alertes-incidents.md
Configuration & tokens	07-configuration.md
Schéma des relations	08-relations.md
Dictionnaire de données	09-dictionnaire-donnees.md

Tables (vue rapide)

Table	Rôle	Lignes estimées
<code>auth_user</code>	Comptes utilisateurs (Django built-in)	< 20
<code>exploitation_invitation_token</code>	Tokens d'invitation à usage unique	= comptes sans mdp
<code>master_token</code>	Tokens d'accès siège → <code>/api/all-data/</code>	1-5
<code>page_permission</code>	Contrôle d'accès par rôle/page	< 50
<code>warehouse</code>	Entrepôts de stockage	< 20
<code>zone</code>	Parcelles d'un entrepôt	< 100
<code>sensor</code>	Capteurs IoT (ESP32/DHT22)	< 100
<code>batch</code>	Lots de café	< 500
<code>movement</code>	Mouvements de lots	< 5 000
<code>measure</code>	Mesures température/humidité	> 100 000
<code>alert_type</code>	Types d'alertes configurables	< 20
<code>alert</code>	Alertes automatiques et manuelles	< 10 000
<code>alert_read</code>	État de lecture des alertes par utilisateur	< 50 000
<code>incident_type</code>	Types d'incidents déclarables	< 20
<code>incident</code>	Incidents déclarés manuellement	< 1 000
<code>parameter</code>	Paramètres de configuration globaux	1

01 — Vue d'ensemble

Rôle de la base de données exploitation

La base de données exploitation est l'instance **locale à chaque plantation**. Elle stocke toutes les données opérationnelles : mesures IoT en temps réel, lots de café, entrepôts, alertes, incidents. Elle est distincte de la BDD siège — aucune FK croisée.

Chaque déploiement exploitation est une instance indépendante. Le siège la consomme via l'endpoint `/api/all-data/` protégé par un master token.

Technologie

Élément	Choix	Justification
SGBD	PostgreSQL 15	Fiabilité, index jsonb, support UUID natif
ORM	Django ORM (migrations)	Schéma versionné, compatible avec le reste du stack
Authentification	Django <code>auth.User</code> + <code>authtoken</code>	Token stateful, rôles gérés via <code>Group</code>
Accès siège	<code>MasterToken</code> custom	Token statique distinct des sessions utilisateurs

Architecture dans le projet

```
[IoT ESP32 – DHT22]
  | MQTT
  ▼
[Broker MQTT]
  | subscribe capteur/iot/+
  ▼
[Simulateur / Worker MQTT]
  | HTTP POST
```



Groupes et rôles

L'exploitation utilise les groupes Django (`auth_group`) pour gérer les accès. Les permissions de page sont stockées en base dans `page_permission`.

Groupe Django	Accès
<code>admin</code>	Toutes les pages, gestion des utilisateurs
<code>responsable_exploitation</code>	Entrepôts, lots, alertes, incidents
<code>responsable_qualite</code>	Mesures, alertes, lots
<code>operateur</code>	Saisie mesures, déclaration incidents

Conventions de modélisation

Convention	Valeur
PK des tables métier	<code>id</code> entier auto-incrémenté
PK des lots	UUID (v4 généré par <code>generate_batch_uuid</code>)

Convention	Valeur
Horodatages	<code>auto_now_add</code> / <code>auto_now</code> ou <code>DateTimeField</code>
FK obligatoire	<code>CASCADE</code> par défaut
FK nullable	<code>SET_NULL</code> , indiqué avec <code>null=True</code> , <code>blank=True</code>
Nom des tables	Explicite via <code>Meta.db_table</code> sur chaque modèle

Schéma de référence

Fichier	Format	Usage
bdd-exploitation/schema-exploitation.png	PNG	Visualisation rapide
bdd-exploitation/schema.puml	PlantUML	Source éditable

02 — Auth & Utilisateurs

Table auth_user (Django built-in)

Django gère les comptes via son modèle `User` standard. L'exploitation n'étend pas `AbstractUser` — elle utilise les groupes pour les rôles.

Colonne	Type	Contrainte	Description
<code>id</code>	int	PK, auto	Identifiant unique
<code>username</code>	varchar(150)	NOT NULL, UNIQUE	Identifiant de connexion
<code>email</code>	varchar(254)	NOT NULL	Email de contact
<code>password</code>	varchar(128)	NOT NULL	Hash PBKDF2 ou ! (inutilisable)
<code>is_staff</code>	boolean	NOT NULL	Accès Django admin
<code>is_active</code>	boolean	NOT NULL, défaut true	Compte actif
<code>is_superuser</code>	boolean	NOT NULL	Toutes permissions
<code>first_name</code>	varchar(150)	NOT NULL, défaut ''	Prénom
<code>last_name</code>	varchar(150)	NOT NULL, défaut ''	Nom
<code>date_joined</code>	timestamptz	NOT NULL	Date de création
<code>last_login</code>	timestamptz	Nullable	Dernière connexion

Groupes (rôles)

Les rôles sont gérés via `auth_group` et la table de liaison `auth_user_groups`.


```
POST /api/auth/login/ → token DRF authtoken
GET  /api/me/          → infos utilisateur + groupes
```

Table exploitation_invitation_token

Même pattern que le siège : token UUID à usage unique pour la première connexion (définition du mot de passe).

Colonne	Type	Contrainte	Description
id	bigint	PK, auto	Identifiant unique
user_id	int	FK → auth_user.id , CASCADE, UNIQUE	Utilisateur cible
token	uuid	NOT NULL, UNIQUE	UUID aléatoire transmis dans le lien
expires_at	timestampz	NOT NULL	Expiration (24 h après création)

Lien d'invitation : `{FRONTEND_URL}/set-password?token={uuid}`

Ré-invitation

Un administrateur peut renvoyer un email d'invitation via :

```
POST /api/users/{id}/resend-invitation/
```

Le token existant est supprimé et un nouveau est créé. L'email est renvoyé avec le nouveau lien.

Table page_permission

Contrôle d'accès basé sur les rôles, stocké en base pour être modifiable sans redéploiement.

Colonne	Type	Contrainte	Description
id	int	PK, auto	Identifiant unique
path	varchar(100)	NOT NULL	Chemin de la page (ex: / dashboard)
role	varchar(50)	NOT NULL	Nom du groupe Django autorisé

Contrainte : `UNIQUE(path, role)` — une même page peut autoriser plusieurs rôles (plusieurs lignes).

Exemples

path	role
/dashboard	admin
/dashboard	responsable_exploitation
/admin/users	admin
/parameters	admin

Flux d'authentification

Invitation :

POST /api/users/ → crée user, génère token, envoie email

POST /set-password/{token}/ → définit le mot de passe, supprime le token

Session :

POST /api/auth/login/ → retourne authtoken DRF dans cookie

HttpOnly

POST /api/auth/logout/ → supprime le token, invalide la session

03 — Entrepôts, Zones & Capteurs

Table warehouse (Entrepôt)

Unité physique de stockage du café. Chaque entrepôt est divisé en zones couvertes par des capteurs IoT.

Colonne	Type	Contrainte	Description
<code>id</code>	int	PK, auto	Identifiant unique
<code>name</code>	varchar(100)	NOT NULL, UNIQUE	Nom de l'entrepôt
<code>surface_m2</code>	float	Nullable	Surface totale en m ² (utilisée pour calculer le nombre de capteurs requis)

Règle de couverture capteur

Le paramètre `sensor_coverage_m2` dans `parameter` définit la surface couverte par un capteur. Le nombre de capteurs recommandés est :

```
ceil(surface_m2 / sensor_coverage_m2)
```

Table zone (Zone)

Parcelle d'un entrepôt assignée à un capteur. Chaque zone correspond à une zone de surveillance physique.

Colonne	Type	Contrainte	Description
<code>id</code>	int	PK, auto	Identifiant unique
<code>warehouse_id</code>	int	FK → <code>warehouse.id</code> , CASCADE	Entrepôt parent

Colonne	Type	Contrainte	Description
<code>index</code>	smallint	NOT NULL	Numéro d'ordre dans l'entrepôt (0, 1, 2...)
<code>name</code>	varchar(10)	NOT NULL	Lettre de zone : A, B, C...
<code>sensor_id</code>	int	FK → <code>sensor.id</code> , SET_NULL, Nullable, UNIQUE	Capteur assigné (OneToOne)

Contrainte : `UNIQUE(warehouse_id, index)` — deux zones d'un même entrepôt ne peuvent pas avoir le même index.

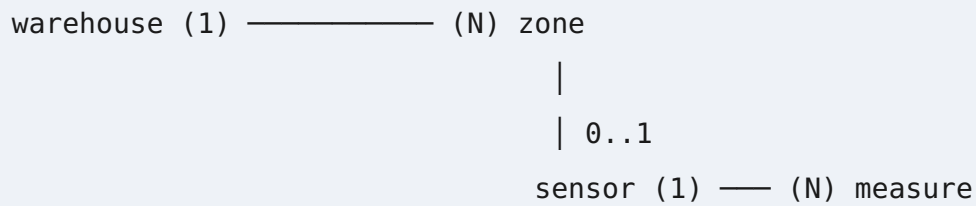
Table sensor (Capteur IoT)

Représente un capteur physique ESP32/DHT22 identifié par adresse IP ou nom.

Colonne	Type	Contrainte	Description
<code>id</code>	int	PK, auto	Identifiant unique
<code>name</code>	varchar(100)	NOT NULL, UNIQUE	Nom du capteur (ex: <code>sensor-01</code>)
<code>ip_address</code>	inet	Nullable, UNIQUE	Adresse IP du module ESP32 sur le réseau local
<code>location</code>	varchar(255)	NOT NULL, défaut , ,	Description de l'emplacement physique
<code>warehouse_id</code>	int	FK → <code>warehouse.id</code> , SET_NULL, Nullable	Entrepôt de rattachement
<code>is_active</code>	boolean	NOT NULL, défaut <code>true</code>	Capteur actif ou désactivé

Un capteur est lié à au plus **une zone** via `zone.sensor_id` (OneToOne inverse).

Relations



- Un entrepôt a plusieurs zones
- Chaque zone a au plus un capteur (0..1)
- Un capteur produit de nombreuses mesures

Flux de découverte capteur

1. Le worker MQTT reçoit un message sur capteur/iot/temperature
2. Il identifie le `sensor_id` dans le payload
3. Si sensor introuvable en base → création automatique (`is_active=true`, `warehouse=null`)
4. L'administrateur assigne ensuite le capteur à un entrepôt + zone via l'interface

04 — Lots & Mouvements

Table batch (Lot)

Un lot représente une quantité de café d'une même variété, récolté à une date donnée.

Colonne	Type	Contrainte	Description
<code>id</code>	<code>uuid</code>	PK	UUID généré par <code>generate_batch_uuid</code> (non modifiable)
<code>creation_date</code>	<code>date</code>	NOT NULL	Date de création du lot (récolte)
<code>expiration_date</code>	<code>date</code>	NOT NULL	Date limite de conservation
<code>variety</code>	<code>varchar(100)</code>	NOT NULL	Variété de café (ex: Arabica, Robusta)
<code>status</code>	<code>varchar(50)</code>	NOT NULL, défaut <code>'active'</code>	Statut : <code>active</code> , <code>archived</code> , <code>shipped</code> ...

Règle de validation

`expiration_date` doit être strictement postérieure à `creation_date`. Cette contrainte est validée au niveau du sérialiseur DRF (pas en base).

Table movement (Mouvement)

Historique des entrées et sorties de lots dans les entrepôts. La clé composite `(warehouse, batch, movement_date)` garantit l'unicité d'un mouvement.

Colonne	Type	Contrainte	Description
<code>id</code>	<code>bigint</code>	PK, auto	Identifiant unique
<code>warehouse_id</code>	<code>int</code>	FK → <code>warehouse.id</code> , CASCADE	Entrepôt concerné
<code>batch_id</code>	<code>uuid</code>	FK → <code>batch.id</code> , CASCADE	Lot concerné
<code>movement_date</code>	<code>timestamptz</code>	NOT NULL	Date et heure du mouvement
<code>movement_type</code>	<code>varchar(50)</code>	NOT NULL	Type : <code>in</code> , <code>out</code> , <code>transfer</code>

Contrainte : `UNIQUE(warehouse_id, batch_id, movement_date)` — un même lot ne peut pas être enregistré deux fois dans le même entrepôt à la même seconde.

Types de mouvements

Type	Signification
<code>in</code>	Entrée du lot dans l'entrepôt
<code>out</code>	Sortie du lot de l'entrepôt
<code>transfer</code>	Transfert depuis un autre entrepôt

Archivage en masse

L'API expose un endpoint pour archiver plusieurs lots en une seule requête :

```
POST /api/batches/bulk-archive/
Body: { "ids": ["uuid1", "uuid2", ...] }
```

Les lots sont passés en `status = "archived"` . Les mouvements restent en base.

Localisation d'un lot

La position actuelle d'un lot n'est pas stockée directement — elle se déduit du dernier mouvement `in` ou `transfer` sans `out` correspondant.

05 — Mesures IoT

Table measure (Mesure)

Enregistrement de chaque relevé de température et d'humidité envoyé par les capteurs IoT via MQTT. C'est la table la plus volumineuse de la base.

Colonne	Type	Contrainte	Description
id	int	PK, auto	Identifiant unique
measure_date	timestamptz	NOT NULL	Horodatage de la mesure (envoyé par le capteur)
temperature	float	NOT NULL	Température en °C
humidity	float	NOT NULL	Humidité relative en %
warehouse_id	int	FK → warehouse.id , CASCADE	Entrepôt source
sensor_id	int	FK → sensor.id , SET_NULL, Nullable	Capteur source (nullable : capteur supprimé conserve les mesures)

Index

Index	Colonnes	Utilisation
Tri par date	-measure_date	Requêtes récentes (/ api/measures/latest/)
Mesures par entrepôt	warehouse_id, -measure_date	Dashboard par entrepôt

Bornes de validation (depuis `sensors.h` ESP32)

Les valeurs hors bornes sont rejetées côté IoT avant publication MQTT. Le backend valide également :

Capteur	Borne basse	Borne haute	Type de borne
Température	-20 °C	60 °C	Exclusive (> / <)
Humidité	0 %	100 %	Inclusive (>= / <=)

Les valeurs `NaN` (capteur DHT22 défaillant) déclenchent la publication sur le topic `capteur/iot/error` et ne sont pas insérées en base.

Flux d'insertion

```
ESP32 DHT22 lecture
|
├─ valide ? → MQTT capteur/iot/temperature ou capteur/iot/
humidity
└─ invalide / NaN → MQTT capteur/iot/error

Worker MQTT
|
├─ subscribe capteur/iot/temperature → POST /api/measures/
├─ subscribe capteur/iot/humidity    → POST /api/measures/
└─ subscribe capteur/iot/error        → log + pas d'insertion
```

Format du payload MQTT

```
{
  "metric": "temperature",
  "value": 22.5,
  "timestamp": "2026-07-02T10:00:00Z",
  "sensor_id": 3
}
```

Volumes et rétention

La table `measure` grossit rapidement : un capteur actif envoie une mesure toutes les N secondes. Sur 15 capteurs à 1 mesure/min : - $15 \times 60 \times 24 =$ **21 600 lignes/jour** - Sur 1 an : ~8 millions de lignes

Aucune politique de rétention automatique n'est configurée actuellement — à prévoir en production.

06 — Alertes & Incidents

Distinction alertes / incidents

	Alerte	Incident
Déclenchement	Automatique (seuil dépassé) ou manuel	Manuel uniquement
Acteur	Système / utilisateur	Utilisateur
Objet	Lié à un entrepôt ou lot	Lié à un entrepôt, lot ou capteur
Type	AlertType	IncidentType
Lecture	Tracée dans alert_read	Non tracée
Table	alert	incident

Table alert_type

Liste configurable des types d'alertes.

Colonne	Type	Contrainte	Description
id	int	PK, auto	Identifiant unique
name	varchar(100)	NOT NULL, UNIQUE	Libellé du type (ex: Température élevée)

Table alert (Alerte)

Colonne	Type	Contrainte	Description
id	int	PK, auto	Identifiant unique
message	text	NOT NULL	Description de l'alerte

Colonne	Type	Contrainte	Description
<code>alert_date</code>	timestamptz	NOT NULL, auto_now_add	Date de création (automatique)
<code>resolved</code>	boolean	NOT NULL, défaut <code>false</code>	Alerte résolue
<code>resolved_date</code>	timestamptz	Nullable	Date de résolution
<code>resolution_comment</code>	text	Nullable	Commentaire de résolution
<code>type_id</code>	int	FK → <code>alert_type.id</code> , CASCADE	Type de l'alerte
<code>warehouse_id</code>	int	FK → <code>warehouse.id</code> , CASCADE, Nullable	Entrepôt concerné
<code>batch_id</code>	uuid	FK → <code>batch.id</code> , CASCADE, Nullable	Lot concerné
<code>created_by_id</code>	int	FK → <code>auth_user.id</code> , SET_NULL, Nullable	Utilisateur créateur (null si système)

Index

Index	Colonnes
Tri par date	<code>-alert_date</code>
Alertes par lot	<code>batch_id</code> , <code>-alert_date</code>
Alertes non résolues	<code>resolved</code> , <code>-alert_date</code>

Résolution

```

alert.resolve(resolution_comment="Température revenue à la normale")
# → resolved=True, resolved_date=now(), resolution_comment="..."

```

Table alert_read

Marque la lecture d'une alerte par un utilisateur. Permet d'afficher un badge "non lu" par utilisateur.

Colonne	Type	Contrainte	Description
<code>id</code>	bigint	PK, auto	Identifiant unique
<code>alert_id</code>	int	FK → <code>alert.id</code> , CASCADE	Alerte lue
<code>user_id</code>	int	FK → <code>auth_user.id</code> , CASCADE	Utilisateur ayant lu l'alerte
<code>read_at</code>	timestamptz	NOT NULL, auto_now_add	Date de lecture

Contrainte : `UNIQUE(alert_id, user_id)` — une alerte ne peut être marquée lue qu'une fois par utilisateur.

Table incident_type

Liste configurable des types d'incidents, ordonnée par `order` pour l'affichage.

Colonne	Type	Contrainte	Description
<code>id</code>	int	PK, auto	Identifiant unique
<code>name</code>	varchar(150)	NOT NULL, UNIQUE	Libellé (ex: Dégât matériel , Contamination)
<code>order</code>	smallint	NOT NULL, défaut <code>0</code>	Ordre d'affichage dans l'interface

Table incident (Incident)

Incident déclaré manuellement par un opérateur. Plus structuré qu'une alerte : lié à un type, peut référencer un capteur.

Colonne	Type	Contrainte	Description
<code>id</code>	int	PK, auto	Identifiant unique
<code>incident_type_id</code>	int	FK → <code>incident_type.id</code> , PROTECT	Type de l'incident — PROTECT empêche la suppression d'un type utilisé
<code>description</code>	text	NOT NULL, défaut ''	Description libre de l'incident
<code>declared_at</code>	timestampz	NOT NULL, auto_now_add	Date de déclaration
<code>resolved</code>	boolean	NOT NULL, défaut <code>false</code>	Incident résolu
<code>resolved_at</code>	timestampz	Nullable	Date de résolution
<code>resolution_comment</code>	text	Nullable	Commentaire de résolution
<code>declared_by_id</code>	int	FK → <code>auth_user.id</code> , SET_NULL, Nullable	Déclarant
<code>resolved_by_id</code>	int	FK → <code>auth_user.id</code> , SET_NULL, Nullable	Résolvant
<code>warehouse_id</code>	int	FK → <code>warehouse.id</code> , CASCADE, Nullable	Entrepôt concerné
<code>batch_id</code>	uuid	FK → <code>batch.id</code> , CASCADE, Nullable	Lot concerné
<code>sensor_id</code>	int	FK → <code>sensor.id</code> , SET_NULL, Nullable	Capteur concerné

La FK sur *incident_type* utilise *PROTECT* (pas *CASCADE*) : on ne peut pas supprimer un type d'incident tant que des incidents y font référence.

07 — Configuration & Tokens

Table parameter (Paramètres globaux)

Une seule ligne en base. Centralise les seuils de surveillance et les préférences du site.

Colonne	Type	Contrainte	Description
id	int	PK, auto	Identifiant unique
exploitation_name	varchar(100)	NOT NULL, UNIQUE, défaut 'Plantation São Paulo'	Nom de la plantation
temperature_tolerance	float	NOT NULL, défaut 2.0	Écart acceptable en °C par rapport à viable_temperature
humidity_tolerance	float	NOT NULL, défaut 5.0	Écart acceptable en % par rapport à viable_humidity
viable_temperature	float	NOT NULL, défaut 20.0	Température cible optimale en °C
viable_humidity	float	NOT NULL, défaut 60.0	Humidité cible optimale en %
temp_calibration_offset	float	NOT NULL, défaut 0.0	Correction systématique appliquée aux températures reçues
humi_calibration_offset	float	NOT NULL, défaut 0.0	Correction systématique appliquée aux humidités reçues

Colonne	Type	Contrainte	Description
<code>alert_notify_roles</code>	<code>varchar(255)</code>	NOT NULL	Groupes à notifier sur nouvelle alerte (séparés par virgule)
<code>incident_notify_roles</code>	<code>varchar(255)</code>	NOT NULL	Groupes à notifier sur nouvel incident
<code>sensor_coverage_m2</code>	<code>float</code>	NOT NULL, défaut <code>4.0</code>	Surface couverte par capteur (pour calcul du nombre requis)

Logique de seuils

Une alerte est déclenchée quand :

```
|mesure - viable_temperature| > temperature_tolerance
```

Soit pour T=20°C, tolérance=2°C : alerte si T < 18°C ou T > 22°C.

Table master_token (Token d'accès siège)

Token statique permettant au siège d'appeler `/api/all-data/` sans session utilisateur. Distinct des tokens de session DRF.

Colonne	Type	Contrainte	Description
<code>id</code>	<code>bigint</code>	PK, auto	Identifiant unique
<code>name</code>	<code>varchar(100)</code>	NOT NULL, UNIQUE	Nom descriptif du token (ex: <code>siege-principal</code>)
<code>token</code>	<code>varchar(64)</code>	NOT NULL, UNIQUE, INDEX	Valeur du token (hexadécimal)
<code>is_active</code>	<code>boolean</code>	NOT NULL, défaut <code>true</code>	Token actif. <code>false</code> = révoqué sans suppression

Colonne	Type	Contrainte	Description
<code>created_at</code>	timestampz	NOT NULL, auto_now_add	Date de création

Génération

```
python manage.py create_master_token --name siege-principal
```

Utilisation (côté siège)

```
GET /api/all-data/  
Authorization: Token <valeur_du_master_token>
```

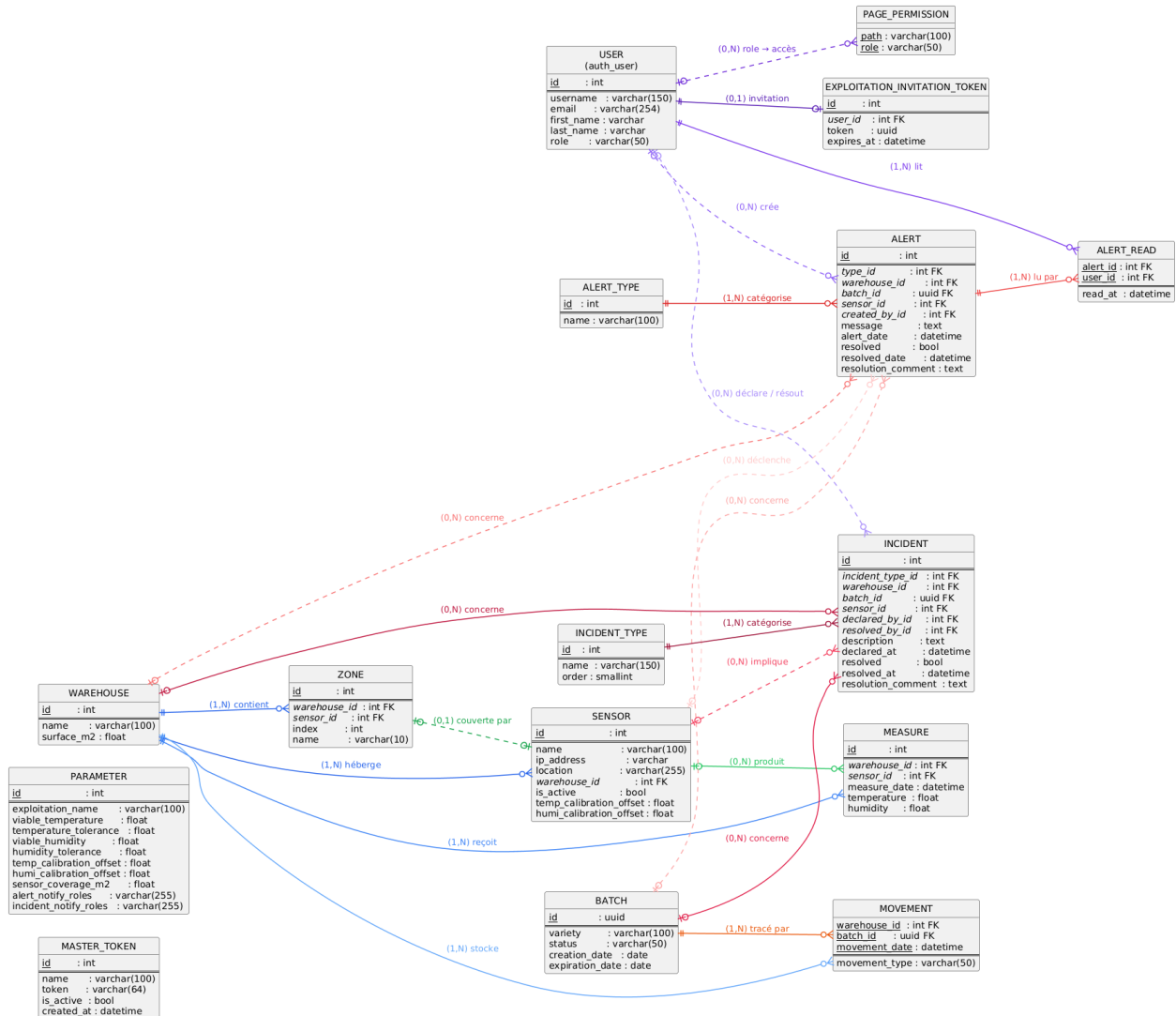
Seul `/api/all-data/` est accessible via master token — les autres endpoints requièrent une session utilisateur.

Sécurité

Élément	Mesure
<code>master_token.token</code>	Valeur aléatoire 64 chars, stockée en clair (utiliser HTTPS en production)
Sessions utilisateurs	Token DRF dans cookie <code>HttpOnly; Secure</code>
Permissions de page	Gérées en base (<code>page_permission</code>) — modifiables sans redéploiement

08 — Schéma des relations

Schéma



Clés étrangères

Table source	Colonne	Table cible	Colonne	On Delete
exploitation_invitation_token	user_id	auth_user	id	CASCADE
zone	warehouse_id	warehouse	id	CASCADE
zone	sensor_id	sensor	id	SET_NULL

Table source	Colonne	Table cible	Colonne	On Delete
sensor	warehouse_id	warehouse	id	SET_NULL
measure	warehouse_id	warehouse	id	CASCADE
measure	sensor_id	sensor	id	SET_NULL
movement	warehouse_id	warehouse	id	CASCADE
movement	batch_id	batch	id	CASCADE
alert	type_id	alert_type	id	CASCADE
alert	warehouse_id	warehouse	id	CASCADE
alert	batch_id	batch	id	CASCADE
alert	created_by_id	auth_user	id	SET_NULL
alert_read	alert_id	alert	id	CASCADE
alert_read	user_id	auth_user	id	CASCADE
incident	incident_type_id	incident_type	id	PROTECT
incident	warehouse_id	warehouse	id	CASCADE
incident	batch_id	batch	id	CASCADE
incident	sensor_id	sensor	id	SET_NULL
incident	declared_by_id	auth_user	id	SET_NULL
incident	resolved_by_id	auth_user	id	SET_NULL

Cardinalités

Relation	Cardinalité	Note
<code>auth_user</code> → <code>exploitation_invitation_token</code>	1 à 0..1	OneToOne — au plus un token par utilisateur
<code>warehouse</code> → <code>zone</code>	1 à N	Un entrepôt a plusieurs zones
<code>zone</code> → <code>sensor</code>	0..1 à 0..1	OneToOne — un capteur par zone maximum
<code>sensor</code> → <code>warehouse</code>	N à 0..1	Un capteur peut être dans un entrepôt
<code>sensor</code> → <code>measure</code>	1 à N	Un capteur produit de nombreuses mesures
<code>warehouse</code> → <code>measure</code>	1 à N	Toutes les mesures d'un entrepôt
<code>batch</code> → <code>movement</code>	1 à N	Historique des déplacements d'un lot
<code>warehouse</code> → <code>movement</code>	1 à N	Mouvements dans un entrepôt
<code>alert_type</code> → <code>alert</code>	1 à N	Un type peut avoir plusieurs alertes
<code>alert</code> → <code>alert_read</code>	1 à N	Une alerte lue par plusieurs utilisateurs
<code>incident_type</code> → <code>incident</code>	1 à N	PROTECT — type non supprimable si utilisé

Stratégies On Delete

Stratégie	Quand	Effet
CASCADE	Suppression d'une entité parent	Supprime toutes les lignes liées
SET_NULL	Suppression d'une entité référencée	Met la colonne à <code>NULL</code> (données conservées)

Stratégie	Quand	Effet
PROTECT	incident_type référéncé	Empêche la suppression du type

CASCADE sur warehouse → measure : si un entrepôt est supprimé, toutes ses mesures le sont aussi. C'est voulu — les mesures sans entrepôt n'ont pas de sens.

Tables indépendantes

Table	Dépend de
parameter	Aucune FK — table singleton
master_token	Aucune FK — token machine
page_permission	Aucune FK vers le modèle Django — utilise des chaînes
alert_type	Aucune FK
incident_type	Aucune FK
auth_user	Modèle Django built-in

09 — Dictionnaire de données

Référence exhaustive de toutes les colonnes de la base de données exploitation.

auth_user (Django built-in)

Colonne	Type PostgreSQL	Null	Défaut	Description
id	int	Non	auto	Clé primaire
username	varchar(150)	Non	—	Identifiant de connexion
email	varchar(254)	Non	—	Email
password	varchar(128)	Non	—	Hash PBKDF2
is_staff	boolean	Non	false	Accès admin Django
is_active	boolean	Non	true	Compte actif
is_superuser	boolean	Non	false	Toutes permissions
first_name	varchar(150)	Non	' '	Prénom
last_name	varchar(150)	Non	' '	Nom
date_joined	timestampz	Non	now()	Date de création
last_login	timestampz	Oui	null	Dernière connexion

exploitation_invitation_token

Colonne	Type PostgreSQL	Null	Défaut	Description
id	bigint	Non	auto	Clé primaire

Colonne	Type PostgreSQL	Null	Défaut	Description
user_id	int	Non	—	FK → auth_user.id (CASCADE, UNIQUE)
token	uuid	Non	uuid4()	Token d'invitation
expires_at	timestampztz	Non	—	Expiration (24 h)

master_token

Colonne	Type PostgreSQL	Null	Défaut	Description
id	bigint	Non	auto	Clé primaire
name	varchar(100)	Non	—	Nom descriptif (UNIQUE)
token	varchar(64)	Non	—	Valeur hexadécimale (UNIQUE, INDEX)
is_active	boolean	Non	true	Token actif
created_at	timestampztz	Non	now()	Date de création

page_permission

Colonne	Type PostgreSQL	Null	Défaut	Description
id	int	Non	auto	Clé primaire
path	varchar(100)	Non	—	Chemin de la page

Colonne	Type PostgreSQL	Null	Défaut	Description
<code>role</code>	<code>varchar(50)</code>	Non	—	Groupe autorisé

Contrainte : `UNIQUE(path, role)`

warehouse

Colonne	Type PostgreSQL	Null	Défaut	Description
<code>id</code>	<code>int</code>	Non	auto	Clé primaire
<code>name</code>	<code>varchar(100)</code>	Non	—	Nom unique
<code>surface_m2</code>	<code>float8</code>	Oui	null	Surface en m ²

zone

Colonne	Type PostgreSQL	Null	Défaut	Description
<code>id</code>	<code>int</code>	Non	auto	Clé primaire
<code>warehouse_id</code>	<code>int</code>	Non	—	FK → <code>warehouse.id</code> (CASCADE)
<code>index</code>	<code>smallint</code>	Non	—	Numéro d'ordre
<code>name</code>	<code>varchar(10)</code>	Non	—	Lettre de zone (A, B...)
<code>sensor_id</code>	<code>int</code>	Oui	null	FK → <code>sensor.id</code> (SET_NULL, UNIQUE)

Contrainte : `UNIQUE(warehouse_id, index)`

sensor

Colonne	Type PostgreSQL	Null	Défaut	Description
id	int	Non	auto	Clé primaire
name	varchar(100)	Non	—	Nom unique
ip_address	inet	Oui	null	IP du module ESP32 (UNIQUE)
location	varchar(255)	Non	' '	Emplacement physique
warehouse_id	int	Oui	null	FK → warehouse.id (SET_NULL)
is_active	boolean	Non	true	Capteur actif

batch

Colonne	Type PostgreSQL	Null	Défaut	Description
id	uuid	Non	generate_batch_uuid()	Clé primaire UUID
creation_date	date	Non	—	Date de récolte
expiration_date	date	Non	—	Date limite
variety	varchar(100)	Non	—	Variété de café
status	varchar(50)	Non	'active'	Statut du lot

movement

Colonne	Type PostgreSQL	Null	Défaut	Description
<code>id</code>	<code>bigint</code>	Non	auto	Clé primaire
<code>warehouse_id</code>	<code>int</code>	Non	—	FK → <code>warehouse.id</code> (CASCADE)
<code>batch_id</code>	<code>uuid</code>	Non	—	FK → <code>batch.id</code> (CASCADE)
<code>movement_date</code>	<code>timestamptz</code>	Non	—	Date du mouvement
<code>movement_type</code>	<code>varchar(50)</code>	Non	—	<code>in</code> , <code>out</code> , <code>transfer</code>

Contrainte : `UNIQUE(warehouse_id, batch_id, movement_date)`

measure

Colonne	Type PostgreSQL	Null	Défaut	Description
<code>id</code>	<code>int</code>	Non	auto	Clé primaire
<code>measure_date</code>	<code>timestamptz</code>	Non	—	Horodatage de la mesure
<code>temperature</code>	<code>float8</code>	Non	—	Température en °C
<code>humidity</code>	<code>float8</code>	Non	—	Humidité en %
<code>warehouse_id</code>	<code>int</code>	Non	—	FK → <code>warehouse.id</code> (CASCADE)
<code>sensor_id</code>	<code>int</code>	Oui	null	FK → <code>sensor.id</code> (SET_NULL)

Index : `(-measure_date)` , `(warehouse_id, -measure_date)`

alert_type

Colonne	Type PostgreSQL	Null	Défaut	Description
<code>id</code>	int	Non	auto	Clé primaire
<code>name</code>	varchar(100)	Non	—	Libellé unique

alert

Colonne	Type PostgreSQL	Null	Défaut	Description
<code>id</code>	int	Non	auto	Clé primaire
<code>message</code>	text	Non	—	Description
<code>alert_date</code>	timestamptz	Non	<code>now()</code>	Date de création
<code>resolved</code>	boolean	Non	<code>false</code>	Résolue
<code>resolved_date</code>	timestamptz	Oui	null	Date de résolution
<code>resolution_comment</code>	text	Oui	null	Commentaire
<code>type_id</code>	int	Non	—	FK → <code>alert_type.id</code> (CASCADE)
<code>warehouse_id</code>	int	Oui	null	FK → <code>warehouse.id</code> (CASCADE)
<code>batch_id</code>	uuid	Oui	null	FK → <code>batch.id</code> (CASCADE)

Colonne	Type PostgreSQL	Null	Défaut	Description
created_by_id	int	Oui	null	FK → auth_user.id (SET_NULL)

Index : (-alert_date) , (batch_id, -alert_date) , (resolved, -alert_date)

alert_read

Colonne	Type PostgreSQL	Null	Défaut	Description
id	bigint	Non	auto	Clé primaire
alert_id	int	Non	—	FK → alert.id (CASCADE)
user_id	int	Non	—	FK → auth_user.id (CASCADE)
read_at	timestampz	Non	now()	Date de lecture

Contrainte : UNIQUE(alert_id, user_id)

incident_type

Colonne	Type PostgreSQL	Null	Défaut	Description
id	int	Non	auto	Clé primaire
name	varchar(150)	Non	—	Libellé unique
order	smallint	Non	0	Ordre d'affichage

incident

Colonne	Type PostgreSQL	Null	Défaut	Description
id	int	Non	auto	Clé primaire
incident_type_id	int	Non	—	FK → incident_type.id (PROTECT)
description	text	Non	' '	Description libre
declared_at	timestampz	Non	now()	Date de déclaration
resolved	boolean	Non	false	Résolu
resolved_at	timestampz	Oui	null	Date de résolution
resolution_comment	text	Oui	null	Commentaire
declared_by_id	int	Oui	null	FK → auth_user.id (SET_NULL)
resolved_by_id	int	Oui	null	FK → auth_user.id (SET_NULL)
warehouse_id	int	Oui	null	FK → warehouse.id (CASCADE)
batch_id	uuid	Oui	null	FK → batch.id (CASCADE)
sensor_id	int	Oui	null	FK → sensor.id (SET_NULL)

parameter

Colonne	Type PostgreSQL	Null	Défaut	Description
id	int	Non	auto	Clé primaire
exploitation_name	varchar(100)	Non	'Plantation São Paulo'	Nom de la plantation (UNIQUE)
temperature_tolerance	float8	Non	2.0	Tolérance température (°C)
humidity_tolerance	float8	Non	5.0	Tolérance humidité (%)
viable_temperature	float8	Non	20.0	Température cible (°C)
viable_humidity	float8	Non	60.0	Humidité cible (%)
temp_calibration_offset	float8	Non	0.0	Correction température
humi_calibration_offset	float8	Non	0.0	Correction humidité
alert_notify_roles	varchar(255)	Non	—	Rôles notifiés (alertes)
incident_notify_roles	varchar(255)	Non	—	Rôles notifiés (incidents)
sensor_coverage_m2	float8	Non	4.0	Surface par capteur (m²)